Weekly Report

Prepared by Shih-Kai Lan | Reported to Professor Zhao and Zhengyu Wu | 13 September 2026

Main conclusion: I completed the repository, runtime, data, playback, reward change, and a 500-iteration diagnostic training run requested by Zhengyu Wu. Five of six mission deliverables are complete. Higher accuracy is not yet proven because the laptop run used one eighth as many parallel environments as the README command, so equal iteration numbers do not represent equal training samples.

Evidence at a glance

Evidence	Observed result	Interpretation
Dataset	9,070 of 9,072 clips accepted	99.98 percent passed validation
Coverage	1,296 grid cells; 4,534 left and 4,536 right swings	Balanced command coverage
Physical gates	IK foot residual max 2.999 mm; ZMP error max 1.000 cm	Generated clips meet the configured limits
Execution	Baseline and tuned smoke tests passed	Both reward configurations run end to end
Long run	Completed at iteration 499; final foot error 0.259 m	Execution is proven; the result is not sample-matched to the reference

Mission Alignment

A direct audit against Zhengyu Wu's Slack instructions

Zhenyu Wu 6:51 PM

Hi, Sky, I have updated the code base, the branch for you is at github.com/ZhenyuWu323/tactile_controller/.../playground, you can clone it down and start to train a robot controller in this branch. The command for the training is

```
uv run python scripts/rl/train.py Mjlab-
Decoupled-Joint-Foot-Mimic-Arm-G1 \
  --env.scene.num_envs 8192 \
  --agent.logger tensorboard \
  --agent.run-name run_name
```

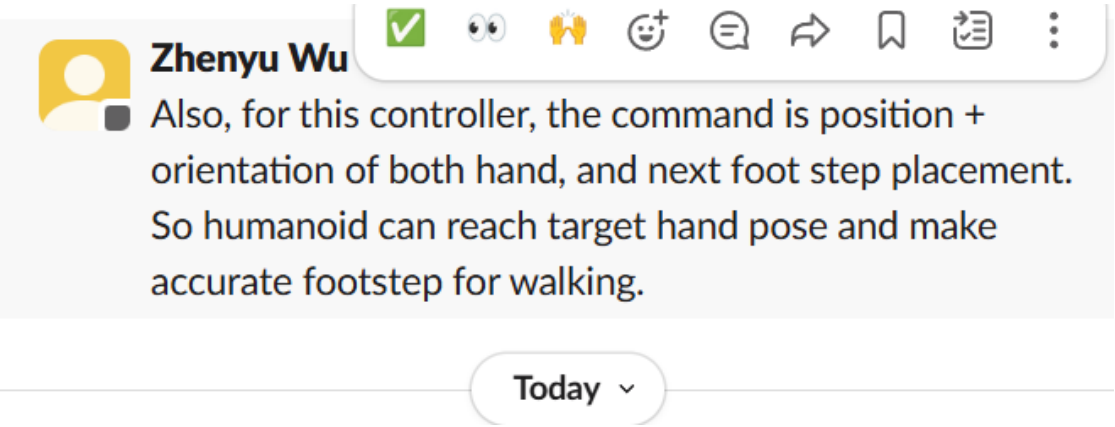
I would recommend to first use AI tools to read through the `README.md` which includes detailed instructions of how to train the controller and how to do some parameter tuning. You can try to tune the parameter especially the reward for foot tracking, to see if it can achieve higher foot step accuracy compared to the one we currently used at `checkpoints/decoupled_footplacement/0907/foot_mimic_noforce_model_34000.pt`. Try to use AI tools to assist you throughout this process, those AI tools can be really good tools for parameter tuning. If you have further questions, you can DM me at anytime.

Also, besides this, I can also send you our experiment code base, and may be you can also work on doing simulation experiment and data analysis if you would like to.

Zhengyu Wu's assignment: train the playground controller and tune foot tracking.

Controller Command

How the upper and lower policies coordinate



Controller input: both hands' position and orientation plus the next foot placement.

The command combines the three-dimensional position and orientation of both hands with the next desired foot placement. The upper-body policy must move the two end effectors toward their targets while the lower-body policy places the foot and maintains balance. Because both policies observe the whole robot state, arm motion and lower-body stabilization can be coordinated rather than treated as independent motions.

Command component	Controlled quantity	Expected behavior
Left hand	Position and orientation	Reach the requested left end-effector pose
Right hand	Position and orientation	Reach the requested right end-effector pose
Next footstep	Foot position and walking target	Place the swing foot accurately without losing balance

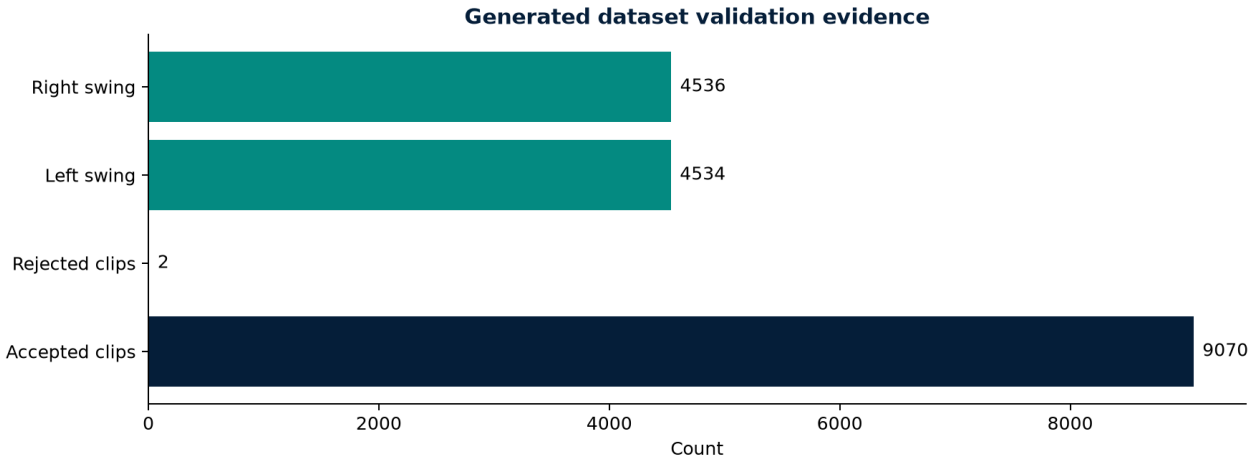
Mission completion audit

hengyu's direction	What I did	Status
se the playground branch	Fast-forwarded feat/playground to commit 21aed05 and created codex/foot-tracking-tuning.	Complete
se AI to study the README	Mapped task registration, data generation, rewards, metrics, checkpoints, and training commands against source.	Complete
ain the controller	Passed two smoke runs and completed a 500-iteration, 1,024-environment TensorBoard run.	Complete
ne foot-tracking parameters	Added one precise foot-position kernel at σ 0.05 m while retaining the σ 0.20 m term.	Complete
se hand pose and footstep mmands	Verified the upper/lower policy interface and played back a 0.15 m forward command.	Complete
chieve higher footstep accuracy	Produced model_499.pt. A fair claim requires a sample-matched baseline or about 4,000 iterations at 1,024 environments, plus playback.	Pending fair proof

Experiment Evidence

Measured outputs, saved artifacts, and interpretation

Generated walking dataset



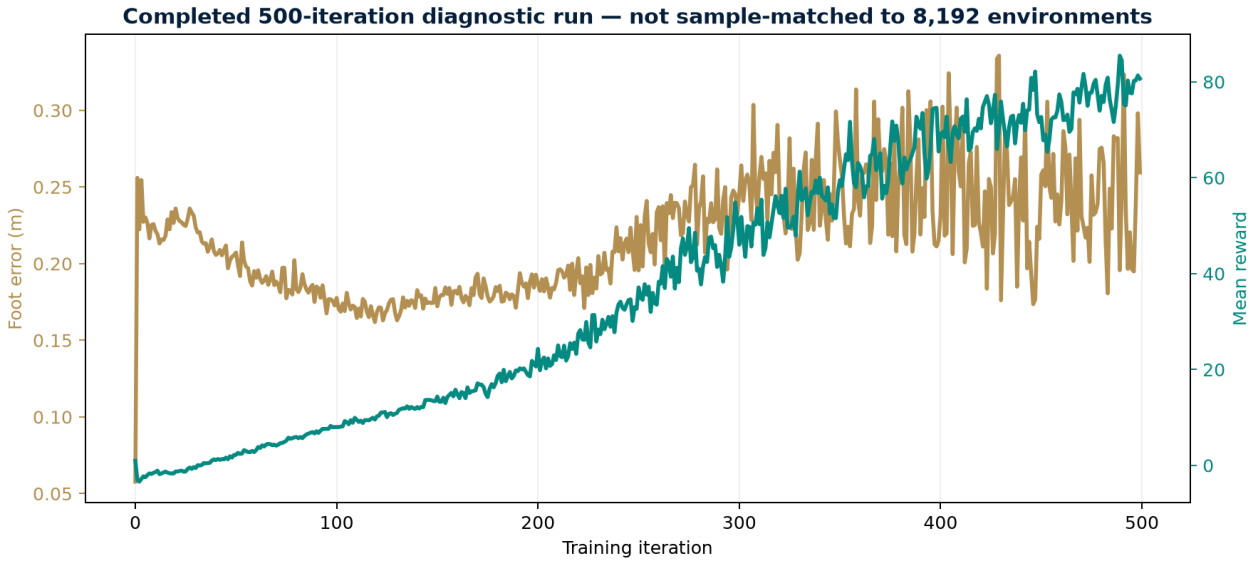
Accepted and rejected candidates plus left and right swing-side balance.

Metric	Result	Why it matters
Acceptance	9,070 of 9,072	Only two clips failed the IK residual gate
Landing error	Maximum 0.000000021 m	Landing target reconstruction is effectively exact
IK foot residual	Maximum 0.002999 m	Below the 3 mm acceptance boundary
IK COM residual	Maximum 0.000515 m	Sub-millimetre COM consistency
ZMP error	Maximum 0.010000 m	At the 1 cm configured bound

Training Evidence

How the run was configured and what the logged metrics show

Training evidence



Completed TensorBoard record through iteration 499. The curve proves execution but is not sample-matched to the 8,192-environment reference.

At the final logged iteration 499, the run reported mean reward 80.66, mean episode length 410.1, foot-position error 0.259 m, and fell-over termination 0.000. Across the final 20 iterations, mean foot error was 0.243 m, mean episode length was 399.0, and mean fell-over termination was 0.000. The saved output is model_499.pt.

Exact run configuration

Parameter	Value	Reason
Task	Mjlab Decoupled Joint Foot Mimic Arm G1	Matches Zhengyu's requested controller
Parallel environments	1,024	Fits the laptop's 8 GB GPU while preserving the same task
Maximum iterations	500	Diagnostic completion gate; not sample-

		equivalent to the 8,192-environment reference
Logger	TensorBoard	Creates persistent, inspectable metric evidence
Run name	foot_precise_s005_500	Records the changed σ value and comparison stage
Seed	42	Supports repeatability across the smoke and tuned tests

Metric definitions

- Foot-position error measures the distance between commanded and achieved foot locations. Lower is better.
- Mean episode length indicates how long policies remain active before termination. Longer values generally indicate improving stability.
- Fell-over termination is a safety guardrail. A lower value is better, but it must be interpreted with the other termination types.
- Mean reward summarizes the configured objective. It is useful for optimization but cannot replace physical error measurements or playback.

Training sample budget

With 24 simulation steps per rollout, 500 iterations at 1,024 environments contain approximately 12,288,000 environment steps. The 8,192-environment command would contain approximately 98,304,000 steps at 500 iterations. Therefore, an approximately sample-matched run on this laptop requires about 4,000 iterations, assuming all other settings remain comparable.

Controlled Foot Accuracy Test

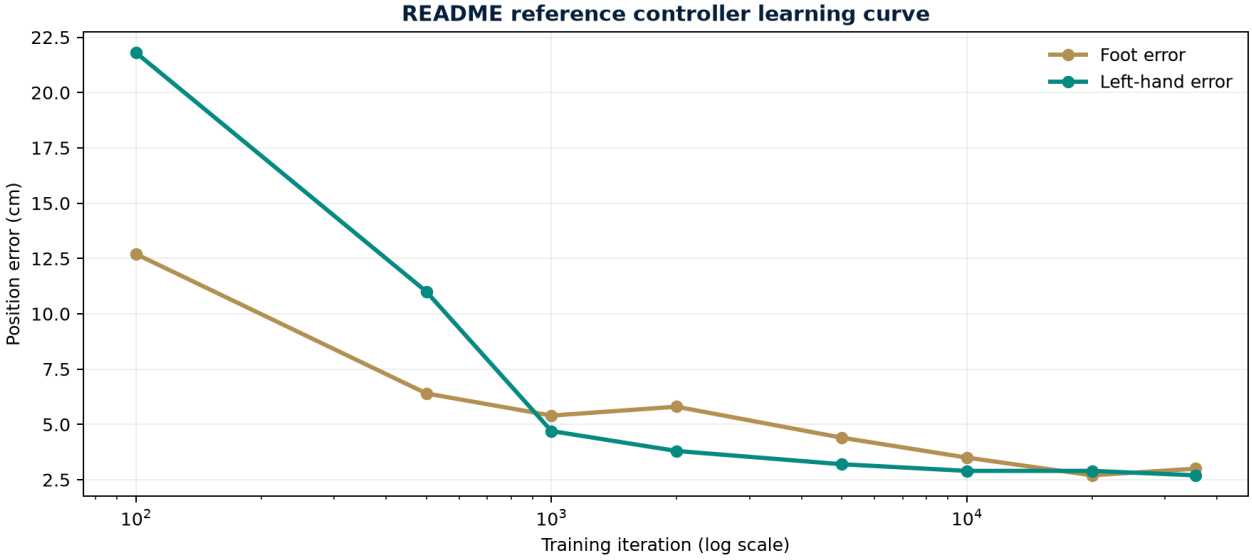
One change, fixed comparison logic

Design element	Baseline	Tuned run
Task	Foot Mimic Arm G1	Same
Seed	42	42
Long-run environments	Reference configuration	1,024 due to 8 GB VRAM
Broad foot reward	σ 0.20 m, weight 1.0	Unchanged
Precise foot reward	None	σ 0.05 m, weight 1.0
Primary metric	Foot-position error	Same physical metric
Guardrails	Falls, episode length, hand errors	Same guardrails

Hypothesis

The existing broad kernel keeps a useful gradient while feet are far from their targets. The added narrow kernel increases discrimination at centimetre scale. The change is accepted only if foot error falls without destabilizing walking or degrading hand tracking.

Reference controller



README metrics for the shipped arm-controller reference. The x-axis is logarithmic.

The README reference reports 0.064 m foot error at iteration 500 and approximately 0.030 m at iteration 35,500. The completed laptop run ended at 0.259 m, with a final-20 mean of 0.243 m. This is numerically higher error, but it is not a fair accuracy verdict because the laptop collected one eighth as many environment samples per iteration. The correct conclusion is that the first diagnostic run executed successfully but did not yet demonstrate higher accuracy.

Comparison procedure

- Run a baseline and tuned controller with the same environment count, seed set, sample budget, and evaluation commands.
- Check foot error together with falls, episode length, hand-position error, and hand-orientation error.
- Run deterministic playback using multiple footstep directions and rotations to catch behaviors hidden by averages.
- At 1,024 environments, extend toward approximately 4,000 iterations to approach the 500-iteration sample budget of an 8,192-environment run.
- Repeat both configurations with additional seeds before treating a difference as a robust improvement.

Engineering Problems and Solutions

The obstacle, the response, and the verification evidence

Problem	How I solved it	Verification
Windows could not resolve the CUDA JAX wheel	Moved the locked Linux environment to Ubuntu WSL instead of changing dependency versions.	JAX returned CudaDevice id 0 on the RTX 5070 Laptop GPU.
The WSL build lacked a compiler and libgomp	Installed user-space Zig compiler tooling and extracted libgomp locally without administrator access.	The 221-package environment synchronized and MJLab imported successfully.
The foot_lip_v2 dataset was missing	Generated all 9,072 candidates and applied the repository's physical validation gates.	9,070 clips passed; statistics were saved to eval_report.json.

Experimental Risks and Solutions

How I prevent a runnable experiment from becoming a misleading conclusion

Problem	How I solved it	Verification
The laptop has 8 GB VRAM	Reduced the long run from 8,192 to 1,024 environments and kept task and rewards unchanged.	Both smoke runs and all 500 iterations completed; model_499.pt was saved.
Reward-only tuning can mislead	Changed one term, fixed the seed, tracked physical error and safety metrics, and required playback.	Telemetry is labeled preliminary until the matched checkpoint and playback are complete.
AI can invent paths or conclusions	Used README and source as authority and required saved evidence for every claim.	Commit, logs, dataset report, code diff, and checkpoints remain traceable.

What I learned from these problems

Each obstacle changed the experimental design. The dependency failure determined the execution platform; the GPU limit changed batch size; the missing dataset required a validation stage; and the risk of reward hacking determined which physical metrics and playback tests must accompany the learning curve. I therefore treat troubleshooting outputs as part of the research evidence, not only as setup work.

Recent Weeks and Summer Progress

Documented work from early August through 13 September

Period	Completed work	Research development
Early August	Built the Unitree G1 rough-terrain Isaac Lab workflow and resumed prior checkpoints.	Learned to preserve training state and separate playback from training.
August	Added a 3 m edge-return behavior and advanced the recorded policy through model_3000.pt.	Connected safety behavior to each environment origin.
Late August	Prepared a 13-file team package and reconnected with the Georgia Tech LIDAR group.	Separated model, source, configuration, logs, and visual evidence.
12–13 September	Moved to tactile_controller, rebuilt the runtime, generated data, verified playback, and launched reward tuning.	Shifted from qualitative demos to controlled metric-based experiments.

Summer summary

- Established a repeatable Unitree G1 reinforcement-learning and simulation workflow.
- Preserved learned behavior by continuing checkpoints instead of restarting training.
- Built and tested a rough-terrain boundary-return safety behavior.
- Packaged models, code, configuration, logs, and visual evidence for team review.
- Transitioned to coordinated manipulation and locomotion using two PPO policies.
- Regenerated a physically validated footstep dataset and began a controlled accuracy experiment.

How I Work With AI

A documented research conversation rather than blind code generation

Stage	What I discuss with AI	My verification responsibility
Grounding	Exact repository, task ID, checkpoint, hardware, README, and error text.	Confirm every path and task name in source.
Hypothesis	Which single parameter should change and what tradeoff should appear.	Reject changes that do not answer the research question.
Small test	The cheapest test that can expose import, memory, or reward wiring failures.	Verify completion and intended metrics.
Evidence	Physical error, episode stability, falls, and deterministic playback.	Do not substitute reward growth for behavior quality.
Communication	How to explain claims, caveats, and remaining gates to the lab.	Keep completed results separate from interpretation.

Next decision

The next experiment should run both baseline and tuned rewards under the same 1,024-environment configuration, preferably across multiple seeds. Alternatively, extend the tuned run toward approximately 4,000 iterations to approach the sample budget of 500 iterations at 8,192 environments. Then perform deterministic playback. Retain the precise reward only if sample-matched evidence supports lower foot error without a material cost to walking or hand tracking.

Evidence Appendix

Files needed to reproduce and inspect the work

Evidence locations

- Dataset report: data/foot_lip_v2/eval_report.json
- Baseline smoke log: logs/rsl_rl/g1_decoupled_joint_foot_mimic_arm/2026-09-13_22-31-58_baseline_smoke_256
- Tuned smoke log: logs/rsl_rl/g1_decoupled_joint_foot_mimic_arm/2026-09-13_22-36-32_foot_precise_s005_smoke_256
- Long run: logs/rsl_rl/g1_decoupled_joint_foot_mimic_arm/2026-09-13_22-38-08_foot_precise_s005_500
- Reward change: src/rl/decoupled/tasks/joint_foot_mimic_arm/env_configs/reward_cfgs.py

Design note: this internal research report uses Georgia Tech's current white, gold, and navy palette and hexagonal visual language. It does not reproduce or modify a restricted official trademark asset.